

Mundo dos Blocos de Tamanho Variável

Representação em LPO, planos de ordem parcial e resolução via SAT

Matheus Henrique de Oliveira Garcia
matheus.garcia@icomp.ufam.edu.br

Samuel de Sousa Castro
samuel.castro@icomp.ufam.edu.br

Gabriel Gregório dos Santos Vitor
gabriel.vitor@icomp.ufam.edu.br

Murillo Rafael de Alcântara Mota
murillo.alcantara@icomp.ufam.edu.br

Lucas Melo dos Santos
lucas.melo@icomp.ufam.edu.br

2 de outubro de 2026

Identificação da equipe. Equipe 20.

Escopo. A solução cobre as três situações do enunciado, inclusive os quatro estados finais alternativos da Situação 1. O relatório apresenta primeiro a representação formal e os planos manuais, depois sua tradução para CNF, execução e comparação com os resultados do resolvedor. A interpretação geométrica e as duas correções necessárias ao exemplo do manual ficam explícitas.

1 Introdução ao problema

O problema consiste em reposicionar quatro blocos de comprimentos diferentes em uma mesa com seis slots. Além da ordem de empilhamento, é necessário representar posições horizontais, níveis verticais, espaços vazios e apoio parcial. O bloco maior pode formar uma ponte, desde que tenha a quantidade de apoio exigida pelo manual.

O enunciado pede representação em lógica de primeira ordem (LPO), ações com pré-condições e efeitos, planos manuais com ordem parcial, tradução para lógica proposicional em CNF, arquivos de entrada e de saída do SAT solver e comparação entre os planos [1]. A estratégia adotada é SATPlan com horizonte limitado: para cada H , uma fórmula representa os estados $0, \dots, H$ e uma ação em cada transição. O primeiro horizonte satisfatível fornece um plano de comprimento mínimo dentro do modelo declarado.

O código e a documentação foram preparados com auxílio de IA, e os planos foram conferidos por simulação geométrica e testes SAT. Os planos de referência abaixo podem ser verificados manualmente; não se atribui a eles uma cronologia de produção humana não documentada.

As imagens foram lidas nas páginas efetivas 5, 6 e 7 do PDF do enunciado. A Situação 1 admite quatro metas alternativas. A Situação 2 leva de S_0 a S_5 . A Situação 3 leva de S_0 a S_7 . Os desenhos intermediários permitem conferir a execução manual. Na Situação 3, S_5 e S_6 têm exatamente as mesmas coordenadas; essa passagem tem zero ações.

2 Descrição formal do mundo dos blocos de tamanho variável

2.1 Objetos e geometria

Usamos um domínio finito com blocos $B = \{a, b, c, d\}$, mesa T , pontos $X = \{0, \dots, 6\}$, slots $S = \{0, \dots, 5\}$ e níveis $L = \{0, 1, 2, 3\}$. O nível zero é a mesa. O comprimento é a função estática

$$\ell(a) = \ell(b) = 1, \quad \ell(c) = 2, \quad \ell(d) = 3.$$

Há sete pontos delimitadores e **seis slots**, cada um representando o intervalo $[s, s + 1]$. Um bloco b começando em p ocupa

$$I(b, p) = \{p, p + 1, \dots, p + \ell(b) - 1\}, \quad 0 \leq p \leq 6 - \ell(b).$$

Assim, a, b têm seis posições válidas, c tem cinco e d tem quatro. A relação estática $\text{overlap}(b, p, y, q)$ vale se $I(b, p) \cap I(y, q) \neq \emptyset$. Blocos que apenas encostam nas bordas não se sobrepõem.

2.2 Predicados e invariantes em LPO

Os fluentes básicos são $\text{at}(b, p, t)$, que fixa o ponto inicial, e $\text{lev}(b, l, t)$, que fixa o nível. Cada bloco possui exatamente uma posição válida e um nível em cada instante:

$$\forall b, t \exists p [\text{at}(b, p, t) \wedge \forall q (\text{at}(b, q, t) \Rightarrow q = p)],$$

com fórmula análoga para lev . Todos os quantificadores são tipados nos conjuntos finitos declarados; nomes distintos denotam objetos distintos e não há outros blocos. Os demais predicados são definidos a partir desses dois:

$$\text{pose}(b, p, l, t) \leftrightarrow \text{at}(b, p, t) \wedge \text{lev}(b, l, t), \tag{1}$$

$$\text{cov}(b, s, l, t) \leftrightarrow \exists p [\text{pose}(b, p, l, t) \wedge s \in I(b, p)], \tag{2}$$

$$\text{occ}(s, l, t) \leftrightarrow \exists b \text{ cov}(b, s, l, t). \tag{3}$$

O predicado de topo livre considera qualquer nível acima, pois não se pode retirar um bloco por uma coluna coberta por outro:

$$\text{clr}(b, t) \leftrightarrow \neg \exists y, p, q, l, m [y \neq b \wedge \text{pose}(b, p, l, t) \wedge \text{pose}(y, q, m, t) \wedge m > l \wedge \text{overlap}(b, p, y, q)]. \quad (4)$$

Os apoios são derivados, inclusive quando há mais de um:

$$\text{on}(b, T, t) \leftrightarrow \text{lev}(b, 0, t), \quad (5)$$

$$\text{on}(b, y, t) \leftrightarrow b \neq y \wedge \exists p, q, l [\text{pose}(b, p, l, t) \wedge \text{pose}(y, q, l - 1, t) \wedge \text{overlap}(b, p, y, q)]. \quad (6)$$

O contato de apoio exige níveis adjacentes. No estado inicial das Situações 1 e 3, d ocupa os slots 3, 4, 5 no nível 1, enquanto a ocupa o slot 3 e b o slot 5 no nível 0. Portanto, $\text{on}(d, a, 0)$ e $\text{on}(d, b, 0)$ valem simultaneamente; o slot 4 está vazio abaixo da ponte.

Dois blocos distintos não podem ocupar o mesmo slot no mesmo nível:

$$\forall b \neq y, s, l, t : \quad \neg(\text{cov}(b, s, l, t) \wedge \text{cov}(y, s, l, t)).$$

Para estabilidade, seja $k_b = \lceil \ell(b)/2 \rceil$. Se $l > 0$, a posição (p, l) de b requer k_b slots distintos cobertos no nível $l - 1$:

$$\text{stable}(b, p, l, t) \leftrightarrow \exists s_1, \dots, s_{k_b} \left[\bigwedge_i (s_i \in I(b, p) \wedge \text{occ}(s_i, l - 1, t)) \wedge \bigwedge_{i < j} s_i \neq s_j \right]. \quad (7)$$

Esta escrita é uma família finita de fórmulas de LPO, uma para cada comprimento; não depende de um operador de contagem primitivo. Todo bloco sobre a mesa é estável, e todo bloco acima dela deve satisfazer a fórmula. a, b, c precisam de um slot de apoio; d precisa de dois. Conta-se a união de slots cobertos, e não a quantidade de blocos de apoio. Essa regra discreta não testa centro de massa, simetria ou física contínua [2].

2.3 Ação e condições de execução

A ação qualitativa é $\text{move}(b, y, p, t)$: mover b para sobre y , começando em p . Se $y = T$, o nível-alvo é $l' = 0$. Se $y \in B \setminus \{b\}$ e $\text{lev}(y, m, t)$, o nível-alvo é $l' = m + 1 \leq 3$, e os spans de b e y devem se sobrepor.

Definimos a coluna de chegada livre, excluindo o próprio bloco movido:

$$\text{free}(b, p, l', t) \leftrightarrow \neg \exists x, q, m [x \neq b \wedge \text{pose}(x, q, m, t) \wedge m \geq l' \wedge \text{overlap}(b, p, x, q)]. \quad (8)$$

São pré-condições: posição válida; $\text{clr}(b, t)$; alvo diferente do próprio bloco; nível-alvo válido; contato com o alvo quando ele é um bloco; coluna de chegada livre; destino diferente da pose atual; e estabilidade do estado resultante. A ação move um bloco por vez, sem transportar os blocos apoiados nele. O deslocamento intermediário é abstrato: o modelo verifica retirada e chegada verticais, sem simular uma trajetória física completa fora da mesa.

Ajuste de interpretação do manual. A exigência literal $\text{clr}(y, t)$ para todo alvo impediria colocar a e b lado a lado sobre c , como exigem as figuras. Por isso, mantemos $\text{clr}(b, t)$ para a origem e usamos free no span local de chegada para o destino. O restante do topo de y pode estar ocupado. A restrição de coluna em todos os níveis superiores é uma escolha conservadora explícita, consistente com a observação do manual de que o espaço sob a ponte está bloqueado. Os resultados se referem a essa interpretação.

2.4 Efeitos e axiomas de persistência

Se b estava em (p_0, l_0) e o destino é (p, l') , os efeitos básicos são:

- **adds:** $\text{at}(b, p, t + 1)$ e $\text{lev}(b, l', t + 1)$;
- **deletes:** a posição e o nível antigos, quando diferentes dos novos;
- **frame:** para cada $x \neq b$, posição e nível persistem.

Formalmente, para todo q, m e $x \neq b$:

$$\text{move}(b, y, p, t) \Rightarrow [\text{at}(x, q, t + 1) \leftrightarrow \text{at}(x, q, t)], \quad (9)$$

$$\text{move}(b, y, p, t) \Rightarrow [\text{lev}(x, m, t + 1) \leftrightarrow \text{lev}(x, m, t)]. \quad (10)$$

Comprimentos, limites e sobreposição geométrica são estáticos e nenhuma ação os altera. pose , cov , occ , clr , on e stable são recalculados a partir de at e lev ; não recebem efeitos independentes. Ao sair uma ponte, seus antigos apoios só ficam livres se nenhum outro bloco continuar acima deles. Ao chegar, a ação torna não livres os blocos cujos spans ficam abaixo dela. Recalcular os predicados evita acrescentar indevidamente $\text{clr}(y)$ para um apoio que ainda sustenta outro bloco.

2.5 Planos de referência e execução manual

Cada par é (p, l) , sempre na ordem a, b, c, d . Escrevemos $b \rightarrow y@p$ para $\text{move}(b, y, p)$ e T para a mesa. As tabelas explicitam estados completos, permitindo verificar espaço, apoio e efeitos a cada passo. O simulador confirma todas as transições. Os números de estado das figuras não devem ser confundidos com instantes SAT.

2.5.1 Plano Q da Situação 3 e meta principal da Situação 1

Passo	Ação	a	b	c	d
0	Início	(3, 0)	(5, 0)	(0, 0)	(3, 1)
1	$d \rightarrow c@0$	(3, 0)	(5, 0)	(0, 0)	(0, 1)
2	$a \rightarrow b@5$	(5, 1)	(5, 0)	(0, 0)	(0, 1)
3	$d \rightarrow T@2$	(5, 1)	(5, 0)	(0, 0)	(2, 0)
4	$a \rightarrow c@0$	(0, 1)	(5, 0)	(0, 0)	(2, 0)
5	$b \rightarrow c@1$	(0, 1)	(1, 1)	(0, 0)	(2, 0)
6	$d \rightarrow T@3$	(0, 1)	(1, 1)	(0, 0)	(3, 0)

$Q = (q_1, \dots, q_6)$ visita $S_0, S_1, S_2, S_3, S_4, S_5, S_7$ da Situação 3. O desenho S_6 repete S_5 : não se insere ação nula. O prefixo (q_1, q_2, q_3, q_4) resolve a Situação 1 até S_{f4} ; o prefixo de cinco ações resolve $S_0 \rightarrow S_5$ da Situação 3.

Correção do manual. O exemplo da página 6 coloca a na mesa em $p = 4$ e depois tenta levar d à mesa em $p = 2$. O span de d inclui o slot 4, ocupado por a : há colisão. Em Q , a ação $q_2 = a \rightarrow b@5$ libera a faixa $[2, 5)$ e corrige a execução até S_{f4} [2].

2.5.2 Plano R da Situação 2 e composição das situações

Passo	Ação	a	b	c	d
0	Início	(0, 1)	(1, 1)	(0, 0)	(3, 0)
1	$b \rightarrow T@2$	(0, 1)	(2, 0)	(0, 0)	(3, 0)
2	$a \rightarrow b@2$	(2, 1)	(2, 0)	(0, 0)	(3, 0)
3	$c \rightarrow d@4$	(2, 1)	(2, 0)	(4, 1)	(3, 0)
4	$a \rightarrow c@4$	(4, 2)	(2, 0)	(4, 1)	(3, 0)
5	$b \rightarrow c@5$	(4, 2)	(5, 2)	(4, 1)	(3, 0)

$R = (r_1, \dots, r_5)$ reproduz exatamente $S_0, \dots, S_5$ da Situação 2. No último passo, a já ocupa o slot 4 sobre c ; b chega ao slot 5, livre. O teste local permite essa configuração.

A referência a “ S_0 até S_5 ” no texto do enunciado admite duas leituras, pois o nome S_0 é reutilizado. Ambas ficam cobertas: R parte do S_0 da Situação 2; partindo do S_0 da Situação 1, a composição Q ; R chega ao mesmo S_5 em 11 ações. Isso funciona porque

$$\text{fim}(Q) = S_7^{(3)} = S_0^{(2)}, \quad \text{fim}(Q; R) = S_5^{(2)} = S_{f2}^{(1)}.$$

A meta S_5 da própria Situação 3 também está coberta pelo prefixo de cinco ações de Q .

2.5.3 Metas alternativas da Situação 1

Para S_{f1} , execute Q e depois $u = a \rightarrow d@4$, $v = b \rightarrow d@5$, $w = c \rightarrow a@4$. Os estados adicionais são:

Passo	Ação	a	b	c	d
7	u	(4, 1)	(1, 1)	(0, 0)	(3, 0)
8	v	(4, 1)	(5, 1)	(0, 0)	(3, 0)
9	w	(4, 1)	(5, 1)	(4, 2)	(3, 0)

u e v podem trocar de ordem. Embora a ação w nomeie a , seu resultado tem $\text{on}(c, a)$ e $\text{on}(c, b)$: o alvo nomeado não exclui apoios adicionais. Para S_{f2} , use $Q; R$, já explicitado.

A alternativa suplementar para S_{f3} abaixo foi refinada com busca e pode ser verificada passo a passo. Ela evita inserir a sob a extremidade suspensa de d :

Passo	Ação	a	b	c	d
0	Início	(3, 0)	(5, 0)	(0, 0)	(3, 1)
1	$d \rightarrow c@0$	(3, 0)	(5, 0)	(0, 0)	(0, 1)
2	$a \rightarrow b@5$	(5, 1)	(5, 0)	(0, 0)	(0, 1)
3	$d \rightarrow T@2$	(5, 1)	(5, 0)	(0, 0)	(2, 0)
4	$a \rightarrow c@1$	(1, 1)	(5, 0)	(0, 0)	(2, 0)
5	$b \rightarrow c@0$	(1, 1)	(0, 1)	(0, 0)	(2, 0)
6	$d \rightarrow T@3$	(1, 1)	(0, 1)	(0, 0)	(3, 0)
7	$a \rightarrow T@2$	(2, 0)	(0, 1)	(0, 0)	(3, 0)
8	$d \rightarrow a@1$	(2, 0)	(0, 1)	(0, 0)	(1, 1)
9	$b \rightarrow T@5$	(2, 0)	(5, 0)	(0, 0)	(1, 1)
10	$d \rightarrow a@0$	(2, 0)	(5, 0)	(0, 0)	(0, 1)

No passo 8, d tem apoio nos slots 1 e 2, fornecidos por c e a . O slot 5 só é liberado depois que d sai de $p = 3$, permitindo o retorno de b . O deslocamento final de d a $p = 0$ produz os apoios em c e a .

Os planos curtos para S_{f1} e S_{f3} que colocam a em $p = 2$ logo após $d \rightarrow c@0$ são inválidos neste modelo: d ainda cobre o slot 2 acima da mesa. De modo semelhante, colocar a em $p = 0$ sob d no plano antigo de S_{f2} viola o corredor. Essas sequências foram substituídas por planos validados, mantendo os estados e objetivos originais.

2.6 Ordem parcial, condições abertas e ameaças

Um plano POP é $P = (A, \prec, C, O)$: ações A , ordem estrita acíclica $\prec$, vínculos causais $A_i \xrightarrow{\varphi} A_j$ e condições abertas O . Incluem-se *Start*, cujos efeitos são os fatos iniciais, e *Finish*, cujas pré-condições são os fatos da meta. Para fechar $(\varphi, A_j) \in O$, escolhe-se um produtor, acrescenta-se o vínculo e a precedência, e abrem-se suas pré-condições ainda não justificadas. Uma ação A_k ameaça o vínculo se pode tornar φ falsa entre produtor e consumidor. Resolve-se por $A_k \prec A_i$ ou $A_j \prec A_k$, ou trocando a ação.

Em Q , os vínculos a seguir explicam a cadeia necessária, sem confundir a cadeia com um algoritmo POP:

Vínculo	Condição fornecida pelo produtor
$q_1 \xrightarrow{\text{clr}(a)} q_2$	retirada da ponte acima de a
$q_2 \xrightarrow{\text{free}(d,2,0)} q_3$	retirada de a do slot 3 da mesa
$q_3 \xrightarrow{\text{free}(a,0,1)} q_4$	retirada de d de cima de c
$q_4 \xrightarrow{\text{clr}(b)} q_5$	retirada de a de cima de b
$q_5 \xrightarrow{\text{free}(d,3,0)} q_6$	retirada de b da faixa de chegada

Fatos de apoio e coordenadas não alterados vêm de *Start* ou da última ação que moveu o bloco. $q_1 \prec \dots \prec q_6$ fecha essas condições. O prefixo até q_4 basta para S_{f4} . Em R , temos $r_1 \xrightarrow{\text{pose}(b,2,0)} r_2$, $r_2 \xrightarrow{\text{clr}(c)} r_3$, $r_3 \xrightarrow{\text{pose}(c,4,1)} r_4$ e $r_4 \xrightarrow{\text{clr}(b)} r_5$. Assim, $r_1 \prec \dots \prec r_5$: antecipar r_5 tentaria retirar b ainda sob a .

Ordem genuinamente parcial. Para S_{f1} , a regressão de *Finish* abre $\text{pose}(c, 4, 2)$, $\text{pose}(a, 4, 1)$, $\text{pose}(b, 5, 1)$ e $\text{pose}(d, 3, 0)$. w, u, v, q_6 são seus produtores. w abre o apoio $\text{pose}(a, 4, 1)$, fornecido por u , e $\text{clr}(c)$, que exige a retirada dos dois blocos de seu topo: u fornece a ausência de a e v a ausência de b . q_6 fornece a posição de d e as faixas de chegada livres para u e v . A ordem resultante é

$$\text{Start} \prec q_1 \prec \dots \prec q_6 \prec \{u, v\} \prec w \prec \text{Finish}.$$

Não se impõe ordem entre u e v . Ambos retiram peças diferentes de c e usam slots distintos de d . w ameaça os vínculos de corredor livre para u e v : se antecipado, criaria um teto sobre os slots 4 e 5. As restrições $u \prec w$ e $v \prec w$ protegem esses vínculos. w também só pode mover c após ambas as retiradas, fechando $\text{clr}(c)$. Os fatos de posição de a, b, d são preservados até *Finish*.

Após expandir as condições derivadas em fatos geométricos, todos os requisitos são fornecidos por *Start* ou por esses produtores; não restam condições abertas. As duas linearizações $Q; u; v; w$ e $Q; v; u; w$ foram reexecutadas no simulador. Isso verifica que o par incomparável é real. A codificação SAT a seguir escolhe **linearizações sequenciais**, com uma ação por instante. Ela não implementa busca POP levantada nem execução paralela; a estrutura parcial é a análise causal declarada aqui.

3 Codificação CNF

3.1 Do domínio finito às variáveis proposicionais

Para um horizonte H , instanciamos os predicados em todos os índices válidos. As famílias principais são $A_{bpt} = \text{at}(b, p, t)$, $L_{blt} = \text{lev}(b, l, t)$, $C_{bt} = \text{clr}(b, t)$ e $M_{bypt} = \text{move}(b, y, p, t)$. Variáveis auxiliares de pose P_{bplt} e ocupação O_{slt} permitem expressar a geometria com cláusulas menores. Instantes de estado variam de 0 a H ; ações variam de 0 a $H - 1$. A relação on é reconstruída depois da solução e não recebe variáveis próprias.

3.2 Cláusulas de estado

Para cada bloco e instante, impomos exatamente uma posição e um nível. Para posição:

$$\bigvee_{p \in \text{Pos}(b)} A_{bpt}, \quad \neg A_{bp_1t} \vee \neg A_{bp_2t} \quad (p_1 < p_2).$$

A regra para L é análoga. A equivalência $P \leftrightarrow (A \wedge L)$ gera

$$(\neg P \vee A), \quad (\neg P \vee L), \quad (P \vee \neg A \vee \neg L).$$

Se duas poses de blocos distintos se sobrepõem no mesmo nível, acrescentamos

$$\neg P_{bplt} \vee \neg P_{yqlt}.$$

Se Q_{slt} é o conjunto de poses que cobrem (s, l) , definimos ocupação por

$$\bigwedge_{P \in Q_{slt}} (\neg P \vee O_{slt}), \quad \neg O_{slt} \vee \bigvee_{P \in Q_{slt}} P.$$

As duas direções impedem um apoio fictício por uma ocupação arbitrariamente verdadeira.

Para estabilidade de uma pose acima da mesa, escrevemos $P_{bplt} \Rightarrow \text{AtLeast}_{k_b} \{O_{s,l-1,t} : s \in I(b, p)\}$. A transformação usada enumera os subconjuntos U de tamanho $\ell(b) - k_b + 1$:

$$\neg P_{bplt} \vee \bigvee_{s \in U} O_{s,l-1,t}.$$

Para c , obtemos $\neg P_{c,p,l,t} \vee O_{p,l-1,t} \vee O_{p+1,l-1,t}$. Para d , são as três cláusulas com os pares de seus três slots. Esse conjunto exige pelo menos dois apoios.

O topo livre equivale à inexistência de uma pose superior sobreposta. Condicional à pose atual $P = P_{bplt}$, para cada bloqueador Q vale $\neg P \vee \neg Q \vee \neg C_{bt}$. A direção inversa é

$$\neg P \vee C_{bt} \vee \bigvee_{Q \in \text{Acima}(b,p,l,t)} Q.$$

Sem bloqueadores possíveis, reduz-se a $\neg P \vee C_{bt}$. Assim, C não pode ser falsificado apenas para facilitar a solução.

3.3 Cláusulas de transição

Cada movimento implica C_{bt} , a posição de destino no próximo estado e o nível determinado pelo alvo. Por exemplo:

$$\neg M_{bTpt} \vee L_{b,0,t+1}, \quad \neg M_{bypt} \vee \neg L_{ytl} \vee L_{b,l+1,t+1}.$$

Para alvo $y \in B$, poses de y sem sobreposição com o destino são proibidas, e $\text{lev}(y, 3, t)$ é incompatível com mover sobre y . A coluna de chegada exclui, com cláusulas condicionais, cada pose de

outro bloco que se sobreponha em nível maior ou igual ao nível-alvo. A ação não pode repetir a pose atual. Os invariantes de estado, incluindo estabilidade, também valem em $t + 1$.

Mais explicitamente, seja $M = M_{bypt}$ e $G = \neg M$ para mesa, ou $G = \neg M \vee \neg L_{y,l-1,t}$ para alvo bloco no nível $l - 1$. O gerador emite:

$$\begin{array}{ll} \neg M \vee C_{bt}, & \neg M \vee A_{bp,t+1} \quad \text{origem livre e posição nova,} \\ G \vee L_{bl,t+1} & \text{nível novo,} \\ G \vee \neg A_{bpt} \vee \neg L_{blt} & \text{proibição de ação nula,} \\ G \vee \neg P_{x,q,m,t} & x \neq b, m \geq l, \text{ overlap}(b, p, x, q). \end{array}$$

Para $y \in B$, $\neg M \vee \neg A_{yqt}$ exclui cada posição q sem contato, e $\neg M \vee \neg L_{y,3,t}$ exclui extrapolação vertical. A estabilidade em $t + 1$ é garantida pelos invariantes, evitando contar como apoio um bloco que está saindo.

Se $\mathcal{M}_{bt}$ contém as ações que movem b , usamos o auxiliar $D_{bt} = \text{moved}(b, t) \leftrightarrow \bigvee_{M \in \mathcal{M}_{bt}} M$, codificado nas duas direções como occ. Para qualquer literal coordenado X (posição ou nível), o frame é

$$(D_{bt} \vee \neg X_t \vee X_{t+1}) \wedge (D_{bt} \vee X_t \vee \neg X_{t+1}).$$

Assim, se b não se move, suas coordenadas persistem. clr, poses e ocupações são recalculados por equivalências, sem frame independente que impediria atualizar a geometria.

Finalmente, em cada $t < H$ há exatamente uma ação: uma cláusula com a disjunção de todas as ações daquele instante e cláusulas binárias proibindo pares distintos. Para $H = 0$, não existe variável de ação nem exigência de movimento. Não há ação de espera.

3.4 Estado inicial, meta e significado de SAT

Posições e níveis do estado inicial são cláusulas unitárias em $t = 0$; a meta usa unitárias em $t = H$. A unicidade torna falsas as alternativas. A fórmula completa é

$$F_H = I_0 \wedge G_H \wedge \bigwedge_{t=0}^H \text{Inv}_t \wedge \bigwedge_{t=0}^{H-1} \text{Trans}_{t,t+1}.$$

Um modelo de F_H determina uma sequência de movimentos legais com exatamente H ações. Reciprocamente, uma sequência legal fornece os valores de posição, nível, auxiliares e ações que satisfazem as cláusulas. Portanto, testar $H = 0, 1, \dots$ até o primeiro SAT prova o comprimento mínimo *nesse domínio e com essas restrições*. Um único resultado UNSAT não prova inexistência de plano para todo horizonte. Como não há espera, não se presume que SAT em H implique SAT em $H + 1$.

4 Exemplos dos três cenários codificados em lógica proposicional

4.1 Instâncias e metas

Os fatos abaixo usam a forma compacta (p, l) ; cada par corresponde às unitárias A_{bpt} e L_{blt} , com $t = 0$ para início e $t = H$ para meta.

Situação	Estado	a	b	c	d
1 e 3	inicial	(3, 0)	(5, 0)	(0, 0)	(3, 1)
1	S_{f1}	(4, 1)	(5, 1)	(4, 2)	(3, 0)
1	S_{f2}	(4, 2)	(5, 2)	(4, 1)	(3, 0)
1	S_{f3}	(2, 0)	(5, 0)	(0, 0)	(0, 1)
1	S_{f4}	(0, 1)	(5, 0)	(0, 0)	(2, 0)
2	inicial	(0, 1)	(1, 1)	(0, 0)	(3, 0)
2	S_5	(4, 2)	(5, 2)	(4, 1)	(3, 0)
3	S_7	(0, 1)	(1, 1)	(0, 0)	(3, 0)

A execução principal da Situação 1 usa S_{f4} , conforme o exemplo do manual. As demais metas são instâncias adicionais executadas e comparadas.

4.2 Cláusulas concretas

No início da Situação 1, temos $A_{a,3,0}$, $L_{a,0,0}$, $A_{d,3,0}$ e $L_{d,1,0}$, além dos fatos de b e c . A meta principal inclui $A_{a,0,H}$, $L_{a,1,H}$, $A_{d,2,H}$ e $L_{d,0,H}$.

A ponte inicial exige dois dos slots 3, 4, 5 ocupados no nível zero. Condicionalmente a $P_{d,3,1,0}$:

$$\begin{aligned} &\neg P_{d,3,1,0} \vee O_{3,0,0} \vee O_{4,0,0}, \\ &\neg P_{d,3,1,0} \vee O_{3,0,0} \vee O_{5,0,0}, \\ &\neg P_{d,3,1,0} \vee O_{4,0,0} \vee O_{5,0,0}. \end{aligned}$$

O estado satisfaz as três cláusulas com ocupações em 3 e 5. Uma pose de a na mesa no slot 4 bloquearia $d \rightarrow T@2$, pois seus spans se intersectam no nível de chegada:

$$\neg M_{d,T,2,t} \vee \neg P_{a,4,0,t}.$$

Na Situação 2, o movimento $c \rightarrow d@4$ implica $A_{c,4,t+1}$ e, como d está no nível zero, $L_{c,1,t+1}$. Na Situação 3, $b \rightarrow c@1$ implica $A_{b,1,t+1}$ e $L_{b,1,t+1}$ quando c está no nível zero. A presença de a sobre c no slot 0 não invalida a chegada no slot 1.

Em DIMACS, cada símbolo recebe um inteiro positivo. A cláusula $(\neg v_i \vee v_j)$ vira a linha `-i j 0`; cada cláusula termina em zero. O cabeçalho `p cnf N M` registra o maior ID e a quantidade de cláusulas. O arquivo `.map` preserva a relação entre IDs e símbolos para decodificar a resposta.

5 Mapeamento da descrição formal para o código

A implementação usa Python 3 e MiniSat22 via PySAT. O gerador não depende do resolvidor: pode produzir DIMACS para um executável MiniSAT externo. Os nomes das funções abaixo são os do código entregue.

Regra ou responsabilidade	Implementação
Domínios, coordenadas e planos	<code>cenarios.py</code> : <code>BLOCKS</code> , <code>FIGURAS</code> , <code>METAS</code> , <code>Q</code> , <code>R</code> , <code>MANUAIS</code>
Símbolos inteiros e DIMACS	<code>Encoding.v</code> , <code>Encoding.add</code> , <code>Encoding.write</code> em <code>bw2cnf_var.py</code>
Unicidade e ação única	<code>Encoding.exactly_one</code> : uma disjunção e exclusões binárias
Disjunções definicionais	<code>Encoding.equivalent_or</code> : ocupação e moved
Pose, colisão, apoio e clear	primeiro laço temporal de <code>encode</code> ; <code>overlap</code> e <code>combinations</code>
Início e meta	unitárias no laço <code>((0, initial), (horizon, goal))</code>
Pré-condições, efeitos e frame	segundo laço temporal de <code>encode</code> ; guardas por nível do alvo
Busca do menor horizonte	<code>resolver.solve</code> : <code>0, 1, ..., Minisat22.solve</code> e modelo
Leitura, CNF e plano	<code>interpretar.py</code> : <code>read_map</code> , <code>read_result</code> , <code>check_cnf</code> , <code>decode</code> , <code>interpret</code>
Validação geométrica independente	<code>dominio.py</code> : <code>valid_state</code> , <code>clear</code> , <code>move</code> , <code>replay</code> , <code>derive_on</code>
Regressões e busca em largura	<code>testes.py</code> : <code>figuras</code> , <code>movimentos</code> , <code>planos</code> , <code>CNF</code> e distâncias BFS

As funções geométricas não consultam cláusulas SAT. Essa segunda implementação permite detectar divergências entre a descrição de ações e a codificação. O mapa é específico de cada CNF; não deve ser trocado entre instâncias ou horizontes.

6 Execução passo a passo

Entre na pasta `TrabalhoIA_MundoBlocos_equipe_X`. Substitua `X` apenas quando o número oficial da equipe for conhecido. Crie um ambiente e instale a dependência fixada:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python resolver.py
```

Sem parâmetros, o resolvidor busca as três metas principais: Situação 1 até S_{f4} , Situação 2 até S_5 e Situação 3 até S_7 . Para reproduzir todas as 16 metas distintas por rótulo:

```
python resolver.py --todos --max-horizonte 14
python testes.py
```

Cada execução principal escreve em `resultados/cenario1`, `cenario2` e `cenario3`. As metas adicionais ficam em subpastas com o nome da meta. Cada diretório contém `trab01_blocos2SAT.cnf`, `trab01_blocos2SAT.map`, `resultadoN.txt`, `plano.txt` e `horizontes.json`. Para gerar uma instância e usar um MiniSAT externo, opcionalmente instalado:

```
python bw2cnf_var.py --cenario 1 --meta Sf4 \
  --horizonte 4 --saida execucao
minisat execucao/trab01_blocos2SAT.cnf execucao/resultado1.txt
python interpretar.py execucao/resultado1.txt
```

O executável MiniSAT costuma retornar código 10 para SAT e 20 para UNSAT; isso não significa falha da resolução. O caminho padrão usa a biblioteca MiniSat22 e dispensa esse executável. O modelo salvo tem formato MiniSAT, com SAT e uma atribuição de inteiros terminada em zero.

LaTeX e Overleaf. `relatorio.tex` é autocontido e não depende dos PDFs de referência nem de figuras externas. Compile duas vezes com pdfLaTeX. No Overleaf, importe esse arquivo e selecione-o como documento principal. O PDF entregue foi compilado e conferido visualmente.

```
pdflatex -interaction=nonstopmode -halt-on-error relatorio.tex
pdflatex -interaction=nonstopmode -halt-on-error relatorio.tex
```

7 Interpretação da saída do SAT solver

7.1 Decodificação e verificação

O resolvidor retorna inteiros, não nomes de blocos. `interpretar.py` lê o mapa, verifica IDs consecutivos e atribuição completa, confere cada cláusula e recupera as ações `mv` verdadeiras em ordem de t . Em seguida, reconstrói as coordenadas de todos os estados e reexecuta cada movimento em `dominio.py`. Se o estado geométrico não coincide com o modelo, a interpretação falha em vez de mostrar um plano não verificado. Finalmente, `derive_on` reconstrói todos os apoios por níveis adjacentes e interseção dos spans.

Por exemplo, no fim da Situação 1 principal: a está sobre c , e b, c, d estão na mesa. No fim da Situação 2: a, b estão sobre c , c sobre d , e d sobre a mesa. No fim da Situação 3: a, b estão sobre c , e c, d estão na mesa. Na ponte inicial, são retornados os dois apoios de d .

7.2 Resultados principais e comparação dos planos

Meta	Referência	Mínimo SAT	Variáveis	Cláusulas
Situação 1 S_{f4}	4	4	1097	37705
Situação 2 S_5	5	5	1334	46706
Situação 3 S_7	6	6	1571	55707

Os horizontes anteriores foram UNSAT: 0 a 3, 0 a 4 e 0 a 5, respectivamente. O SAT reproduziu exatamente o prefixo $Q_{1:4}$ e Q nas Situações 1 e 3. Na Situação 2 encontrou a alternativa ótima

$$(b \rightarrow d@3), (a \rightarrow b@3), (c \rightarrow d@4), (a \rightarrow c@4), (b \rightarrow c@5).$$

Ela estaciona b sobre d , em vez de colocá-lo na mesa em $p = 2$. c chega aos slots 4 e 5 sobre d , sem colidir com b no slot 3. Ambas as sequências têm cinco ações e chegam à mesma meta; estados intermediários de uma solução SAT não precisam coincidir com todos os desenhos.

7.3 Todas as metas desenhadas e limites da comparação

Situação	Meta	Referência explícita	Mínimo SAT
1	S_{f1}	9	9
1	S_{f2}	11 ($Q; R$)	10
1	S_{f3}	10	10
1	S_{f4}	4	4
2	S_1, S_2, S_3, S_4, S_5	1, 2, 3, 4, 5	1, 2, 3, 4, 5
3	S_1, S_2, S_3, S_4	1, 2, 3, 4	1, 2, 3, 4
3	S_5, S_6, S_7	5, 5, 6	5, 5, 6

Os prefixos de R e Q fornecem as referências das metas intermediárias. Há 16 rótulos de meta testados, embora alguns denotem a mesma geometria. Cada instância guarda em `horizontes.json` todos os testes de zero até o primeiro SAT. Com metas completas, os tamanhos são $149 + 237H$

variáveis e $1701 + 9001H$ cláusulas; por exemplo, $H = 9$ tem 2282 variáveis e 82710 cláusulas, e $H = 10$ tem 2519 e 91711.

Para S_{f1} , o SAT encontra a sequência $Q; u; v; w$ de nove ações. Para S_{f3} , encontra a mesma geometria da referência de dez ações; no passo 8 nomeia c como alvo em vez de a , mas ambos são apoios válidos naquela pose. A diferença é simbólica, não geométrica.

Para S_{f2} , a composição $Q; R$ não é ótima. A sequência SAT de dez ações é:

Passo	Ação	Passo	Ação
1	$c \rightarrow T@1$	6	$b \rightarrow a@0$
2	$d \rightarrow c@0$	7	$d \rightarrow T@3$
3	$a \rightarrow b@5$	8	$c \rightarrow d@4$
4	$d \rightarrow c@1$	9	$b \rightarrow c@5$
5	$a \rightarrow T@0$	10	$a \rightarrow c@4$

Mover primeiro c cria outra organização temporária; o plano evita passar pelo estado inicial da Situação 2 e economiza uma ação. As sequências completas e seus estados estão nos arquivos `plano.txt` correspondentes.

Verificação cruzada. Os testes reexecutam referências, validam todas as figuras, rejeitam a colisão do manual e a inserção sob ponte, conferem modelos e mapas, testam as duas ordens de u, v e comparam os 16 mínimos SAT com busca em largura no simulador. Também confrontam cada ação candidata, legal ou ilegal, nos estados desenhados, com o SAT de uma transição. Esses testes fortalecem a validação; não substituem a declaração das hipóteses geométricas.

Conclusão. Posição, nível e ocupação por slots tornam explícitos o apoio parcial e o espaço local. Os três objetivos principais exigem 4, 5 e 6 movimentos no modelo adotado. A análise POP explica quais precedências são necessárias e identifica um par realmente permutável. A composição intuitiva de 11 ações para S_{f2} é válida, mas o SAT encontra uma solução de dez. A otimalidade vale para o domínio discretizado e a movimentação por corredor livre; não é uma alegação sobre toda interpretação física possível dos desenhos.

Referências

- [1] Edjard Mota. *Fundamentos de Inteligência Artificial: 1º Trabalho, representação do conhecimento e raciocínio automatizado para gerar plano que empilha blocos de diferentes dimensões*. Instituto de Computação, UFAM, 2026. Arquivo fornecido: `T1_MundoDosBlocos.pdf`, especialmente pp. 3–7.
- [2] *Manual do Mundo dos Blocos de Tamanho Variável*. Material fornecido com a atividade, arquivo `manual_variable_blocks.pdf`, pp. 2–8. O exemplo de execução da p. 6 é corrigido neste relatório.
- [3] Edjard Mota. *Representação de Conhecimento e Inferência: Ontologia do Jogo da Velha e Revisão dos Métodos*. IComp, UFAM, 22 de setembro de 2026. Material de revisão fornecido.